Informe Actividad 2

Laberinto de busqueda con BFS, DFS Y UCS

Integrantes:

Michael Stevens Cardenas Urbano (2610304)
Luis Santiago Arenas Hincapié (2435300)
Brigitte Vanesa Chavez Riascos(2510107)

Asignatura:
Inteligencia Artificial

Docente:
Andres Mauricio Valencia Restrepo

Facultad de Ingeniería
Escuela de Ingeniería de Sistemas y Computación
Ingeniería de Sistemas

Santiago de Cali, 17 de octubre del 2026

1. Introducción

En esta actividad se resolvió de forma automática el laberinto trabajado en la Actividad 1. Para ello, el laberinto se transformó en un **grafo**: cada casilla por la que se puede caminar es un nodo, y dos nodos están conectados si las casillas son vecinas.

Sobre ese grafo se implementaron tres algoritmos de búsqueda clásicos:

- **BFS** (Búsqueda en Anchura).
- **DFS** (Búsqueda en Profundidad).
- **UCS** (Búsqueda de Costo Uniforme).

El objetivo es encontrar un camino desde el inicio hasta la meta y comparar cómo se comporta cada algoritmo: cuántos pasos da la ruta, cuánto cuesta y cuántos nodos tuvo que revisar para encontrarla.

2. El laberinto

El laberinto original es una cuadrícula de 9×9, donde cada número representa un tipo de casilla:

Valor	Significado	Costo
0	Camino libre	4
1	Pared (bloqueado)	–
2	Premio	1
3	Penalización	12
9	Meta	4

Cuadro 1: Tipos de casilla y su costo.

0	0	0	0	3	0	0	1	0
0	1	1	1	0	1	0	1	0
0	0	0	1	0	1	0	0	0
1	1	2	0	0	1	1	1	0
0	0	0	1	0	0	0	1	0
0	1	0	1	0	1	0	1	0
0	1	0	2	0	1	3	**9**	2
0	1	1	1	3	2	2	1	2
0	0	1	0	0	1	2	2	2

Cuadro 2: Matriz del laberinto original.

Los premios hacen que pasar por una casilla sea más barato, mientras que las penalizaciones lo hacen más caro. Esto es importante porque no siempre el camino más corto es el más barato.

3. Implementación

3.1. Representación del grafo

En lugar de trabajar con la matriz directamente, el laberinto se escribió como una **lista de adyacencia**. Es un diccionario en el que cada nodo tiene la lista de sus vecinos y el costo de moverse hacia cada uno de ellos:

```
grafo = {
    "A": [("I", 4), ("B", 4)],
    "B": [("A", 4), ("C", 4)],
    "C": [("B", 4), ("D", 4)],
    ...
}
```

Cada nodo se nombra con letras (`A`, `B`, ..., `Z`, `AA`, `AB`, ...). El inicio es el nodo `A` (esquina superior izquierda) y la meta es el nodo `AO`.

Elegimos esta forma porque permite **ver y modificar cada costo a mano**: si queremos que un paso sea más caro o más barato, basta con cambiar el número en el diccionario.

3.2. Los algoritmos

Los tres algoritmos funcionan con la misma idea: guardan una lista de nodos pendientes por revisar, marcan los que ya visitaron y recuerdan desde qué nodo llegaron a cada uno, para poder reconstruir el camino al final. Lo que cambia es **cuál nodo eligen revisar a continuación**.

- **BFS** usa una *cola*: revisa primero los nodos que llegaron antes. Así avanza "por capas", como una onda que se expande desde el inicio. Por eso siempre encuentra el camino con **menos pasos**, aunque no mira los costos.
- **DFS** usa una *pila*: revisa primero el último nodo que agregó. Así se mete por un camino hasta el fondo y solo retrocede cuando no puede seguir. Suele encontrar una ruta rápido, pero **no garantiza** que sea la más corta ni la más barata.
- **UCS** siempre revisa el nodo al que es **más barato llegar** hasta ese momento. Por eso sí tiene en cuenta los costos y encuentra el camino de **menor costo total**.

3.3. Funcionamiento del programa

El programa se ejecuta desde la consola y sigue estos pasos:

1. Muestra un menú para elegir el algoritmo: BFS, DFS o UCS.
2. Si se elige BFS o DFS, pregunta si se quiere ejecutar **con costos** o **sin costos**. En el modo con costos se muestran los pesos de cada conexión y el costo total de la ruta; en el modo sin costos solo se muestran la ruta y los pasos. UCS no hace esta pregunta porque siempre necesita los costos.
3. Imprime en consola la ruta encontrada, el número de pasos, el costo (si aplica) y los nodos que se exploraron.

4. Abre una ventana con el dibujo del grafo, donde se resalta la ruta.

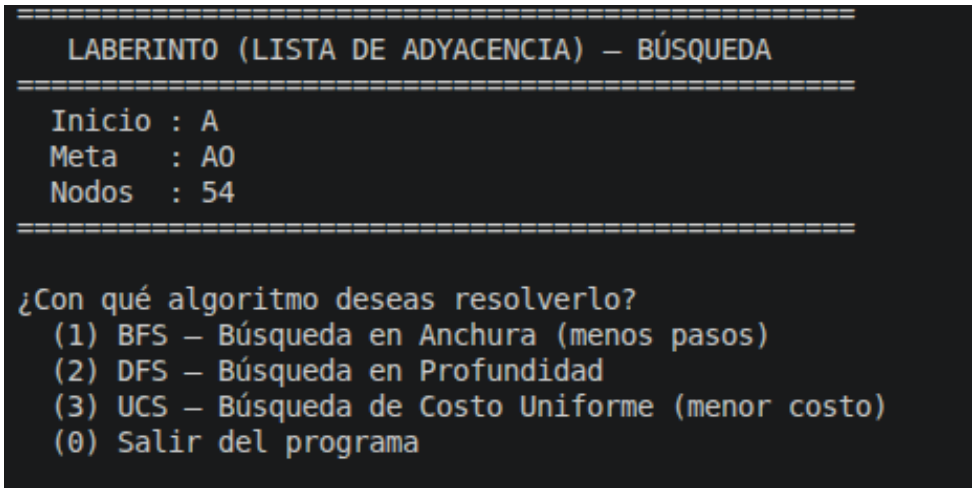


Figura 1: Menú del programa en la consola.

Para el dibujo se usaron las librerías `networkx` y `matplotlib`, únicamente para la parte visual; los algoritmos de búsqueda se programaron a mano. Los colores del dibujo significan lo siguiente:

- **Naranja:** nodo de inicio.
- **Rojo:** meta alcanzada.
- **Verde:** nodos y conexiones que forman la ruta encontrada.
- **Azul:** nodos que el algoritmo revisó pero no quedaron en la ruta.
- **Gris:** nodos que nunca se visitaron.

4. Resultados

4.1. BFS

BFS encontró la ruta con menos pasos (13), pero tuvo que revisar casi todo el laberinto (50 nodos) y la ruta pasa por una penalización, por lo que su costo fue de 65.

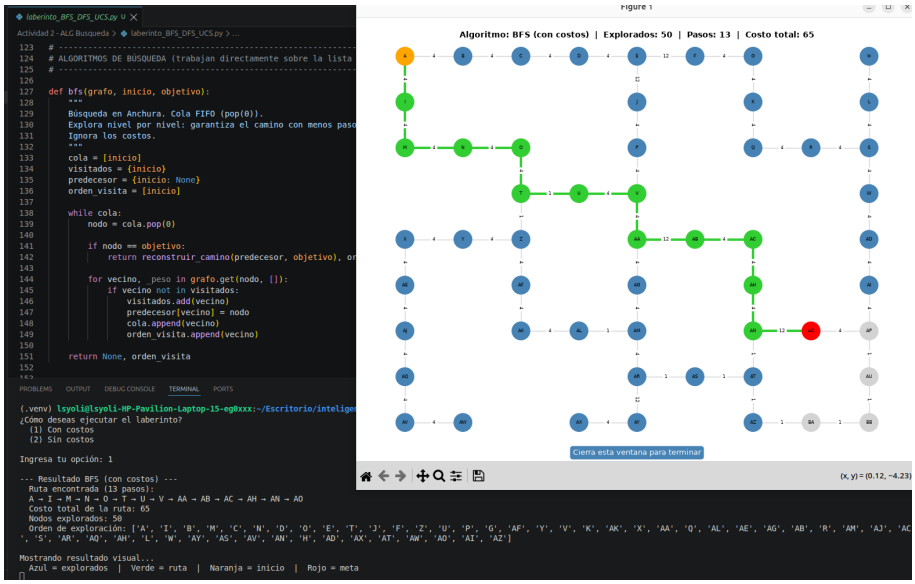


Figura 2: Resultado de BFS con costos.

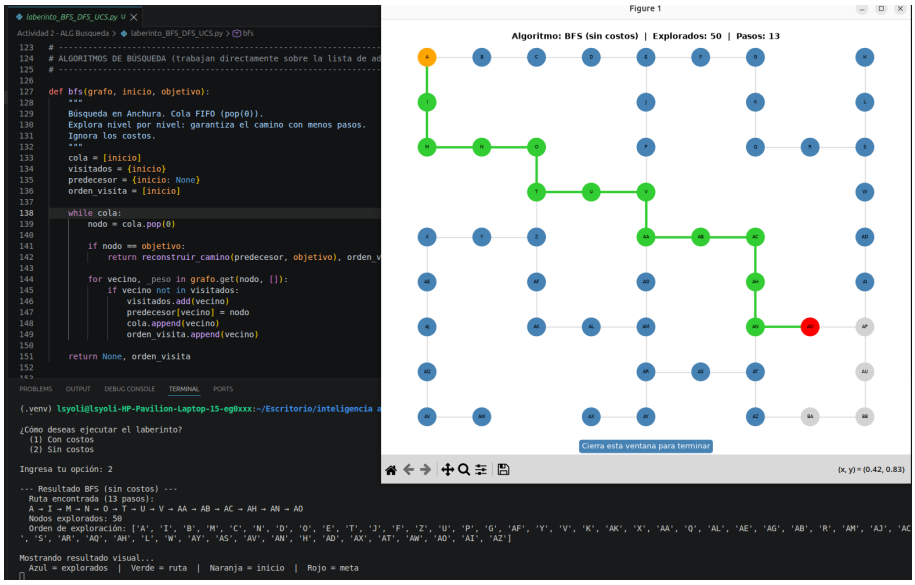


Figura 3: Resultado de BFS sin costos (mismo camino, sin mostrar los pesos).

4.2. DFS

DFS fue el que menos nodos revisó (solo 20), porque se fue directo por un camino hasta llegar a la meta. Sin embargo, la ruta que encontró es más larga (15 pasos) y tampoco es la más barata (costo 65).

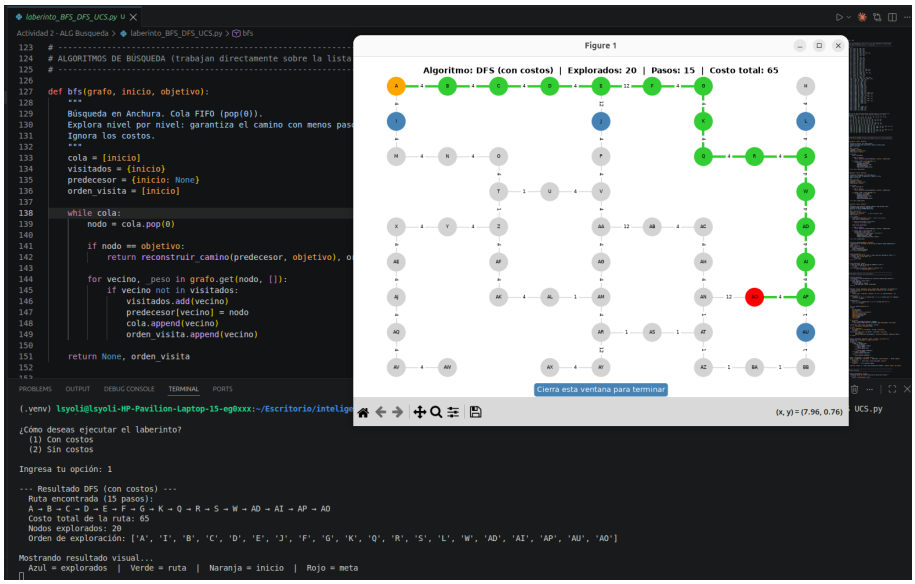


Figura 4: Resultado de DFS con costos.

4.3. UCS

UCS encontró una ruta de 15 pasos, dos más que BFS, pero con el **menor costo de todos: 53**. Esto se debe a que la ruta aprovecha varias casillas de premio y evita la penalización que está junto a la meta.

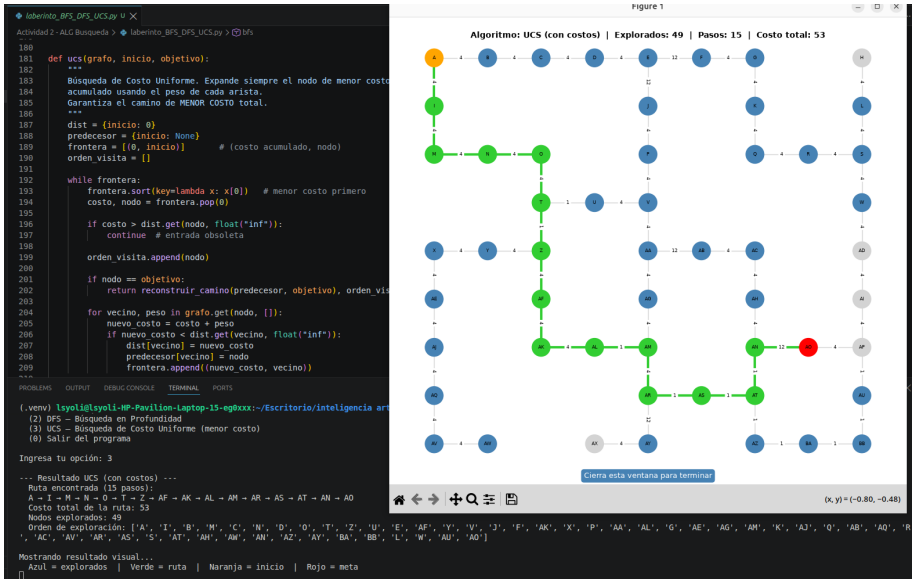


Figura 5: Resultado de UCS.

4.4. Comparación

Algoritmo	Pasos	Costo total	Nodos explorados
BFS	**13**	65	50
DFS	15	65	**20**
UCS	15	**53**	49

Cuadro 3: Comparación de los tres algoritmos.

Las rutas encontradas fueron:

- **BFS:** A → I → M → N → O → T → U → V → AA → AB → AC → AH → AN → AO
- **DFS:** A → B → C → D → E → F → G → K → Q → R → S → W → AD → AI → AP → AO
- **UCS:** A → I → M → N → O → T → Z → AF → AK → AL → AM → AR → AS → AT → AN → AO

5. Conclusiones

- Representar el laberinto como un grafo permitió aplicar algoritmos de búsqueda de forma general: los mismos algoritmos funcionarían con cualquier otro laberinto con solo cambiar la lista de adyacencia.
- **No existe un algoritmo "mejor" en todo**; cada uno responde a una pregunta distinta. BFS responde "¿cuál es el camino más corto?", UCS responde "¿cuál es el camino más barato?" y DFS simplemente busca "algún camino", normalmente de forma rápida.
- El experimento mostró que **el camino más corto no siempre es el más barato**: BFS dio menos pasos, pero UCS encontró una ruta más económica al aprovechar los premios y esquivar las penalizaciones.
- DFS fue el más eficiente en cantidad de nodos revisados, pero su resultado depende mucho del orden en que se recorren los vecinos, por lo que no se puede confiar en que su ruta sea buena.
- Cuando el problema tiene costos (como este laberinto con premios y penalizaciones), UCS es la opción adecuada. Cuando todos los movimientos valen lo mismo, BFS es suficiente y más sencillo.
- La parte visual ayudó a entender cómo "piensa" cada algoritmo: en las figuras se nota que BFS y UCS se expanden por casi todo el laberinto, mientras que DFS sigue un solo camino.